Programa en ensamblador

Snake Game

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Arquitectura de Computadoras Semestre: IV Código: 1702227

Profesor	Proyecto	Duración
Pedro A. Rodriguez Gonzales	Snake Game	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	20/10/2026	10/11/2026

Índice

1. Actividades **4**
1.1. Descripción de la actividad . . . 4
1.2. Objetivos . . . 4
1.3. Entorno de desarrollo y herramientas . . . 4

2. Marco teórico **4**
2.1. Arquitectura 8086 y segmentación . . . 4
2.2. Modelo de memoria del formato COM . . . 5
2.3. Modo texto 80x25 y servicios BIOS (INT 10h) . . . 5
2.4. Lectura de teclado y temporización . . . 5
2.5. Cola circular y direccionamiento indirecto . . . 5

3. Diseño del juego **6**
3.1. Reglas y controles . . . 6
3.2. Representación del tablero . . . 6
3.3. Estructuras de datos . . . 6
3.4. Procedimientos y sus contratos . . . 6

4. Desarrollo e implementación **8**
4.1. Altas y bajas del diseño . . . 8
4.2. Decisiones de implementación destacadas . . . 8
4.3. Tabla de rastreo: Práctica del curso a código . . . 9

5. Funcionamiento **10**
5.1. Bucle principal . . . 10
5.2. Flujo detallado de una tecla . . . 10
5.3. Flujo detallado de un movimiento . . . 11
5.4. Flujo detallado de “comer” . . . 12
5.5. Flujo detallado de un choque . . . 12

6. Pseudocódigo **13**
6.1. Bucle principal . . . 13
6.2. INIT_GAME . . . 13
6.3. DRAW_BORDER . . . 13
6.4. READ_KEY . . . 14
6.5. CHECK_COLLISION . . . 14
6.6. MOVE_SNAKE . . . 14
6.7. SPAWN_FOOD . . . 15
6.8. DRAW_CELL . . . 15
6.9. PRINT_NUM . . . 15
6.10. DELAY . . . 15
6.11. GAME_OVER . . . 16
6.12. Pseudocódigo del programa completo . . . 16

7. Análisis de rendimiento **16**
7.1. ¿Por qué el juego es “lento”? . . . 16
7.2. Costo por fotograma . . . 16
7.3. Cuellos de botella y latencia de entrada . . . 16

8. Limitaciones y consideraciones **17**
8.1. Limitaciones funcionales del juego 17
8.2. Limitaciones técnicas del código 17
8.3. Limitaciones de portabilidad y entorno 18
8.4. Posibles mejoras futuras 18

9. Pruebas y validación **18**
9.1. Procedimiento de prueba recomendado 18

10. Anexos **19**
10.1. Código fuente completo (snakeGame.asm) 19
10.2. Tabla de variables y estructuras 30
10.3. Tabla de servicios BIOS/DOS usados 30
10.4. Referencias 31

1. Actividades

1.1. Descripción de la actividad

Implementación del clásico juego *Snake* para procesador 8086, escrita para el emulador EMU8086 y ensamblada como programa de formato COM.

1.2. Objetivos

El objetivo general es construir un juego funcional en ensamblador 8086 y documentarlo con rigor técnico. Los objetivos específicos son los siguientes:

- Aplicar la *Práctica 2* (sumador total): operaciones aritméticas y lógicas básicas con acarreo, `ADD`, `ADC`, `SUB`, `XOR`, `AND` y desplazamientos.
- Aplicar la *Práctica 3* (modos de direccionamiento): direccionamiento inmediato, de registro, directo e indirecto por registro.
- Aplicar la *Práctica 4* (control de flujo): comparaciones, saltos condicionales e incondicionales, bucles con `LOOP` y llamadas a procedimientos con `CALL`/`RET`.
- Aplicar la *Práctica 5* (operaciones aritméticas y pila): `MUL`, `DIV`, `INC`, `DEC`, `PUSH`, `POP`, constantes con `EQU` y datos con `DB`/`DW`.
- Utilizar los servicios del BIOS (`INT 10h`, `INT 16h`, `INT 1Ah` y `INT 15h`) y de DOS (`INT 21h`) para pantalla, teclado, inicialización de semilla y temporización.
- Documentar el diseño, el funcionamiento, el rendimiento y las limitaciones del programa para su exposición en clase.

1.3. Entorno de desarrollo y herramientas

El desarrollo y la verificación se realizan con las herramientas de la Tabla 1.

Cuadro 1: Herramientas empleadas en el proyecto.

Herramienta	Uso
EMU8086	Edición, ensamblado y emulación del programa en formato COM.
Editor de texto	Edición alternativa del código fuente `snakeGame.asm`.

2. Marco teórico

2.1. Arquitectura 8086 y segmentación

El procesador 8086 es un microprocesador de 16 bits con bus de direcciones de 20 bits. Como sus registros solo almacenan 16 bits, la dirección física se forma combinando un registro de segmento con un desplazamiento (*offset*): la dirección física equivale al segmento multiplicado por 16 más el desplazamiento. Los registros de segmento son `CS` (código), `DS` (datos), `SS` (pila) y `ES` (extra).

Las banderas del registro de estado resultan esenciales para este proyecto. La bandera `ZF` (*Zero Flag*) indica que el resultado de la última operación fue cero o que una comparación resultó en igualdad. La bandera `CF` (*Carry Flag*) indica un acarreo o un préstamo en operaciones sin signo. Estas dos banderas deciden todos los saltos condicionales del programa (véase la Tabla 2).

Las instrucciones `JL`/`JGE`, que interpretan los operandos con signo, no se utilizan en este código, por la razón que explica la sección 2.2.

Cuadro 2: Banderas que deciden los saltos condicionales del código.

Salto	Bandera	Interpretación
`JE` / `JNE`	`ZF`	Igualdad o desigualdad tras `CMP`.
`JB` / `JAE`	`CF`	Comparación *sin signo*: menor que / mayor o igual que.

2.2. Modelo de memoria del formato COM

Un programa COM se carga en un único segmento y ocupa como máximo 64 KB. El cargador de DOS inicializa `CS`, `DS`, `ES` y `SS` con el mismo valor, de modo que el programa no necesita inicializar `DS` manualmente: código, datos y pila comparten segmento desde el arranque. Por esa razón el programa no declara `.model` ni `.stack`, y comienza con la directiva `org 100h`, que reserva los primeros 256 bytes del segmento para el prefijo de segmento de programa (PSP).

Comparación con signo y sin signo. Tras `CMP`, las instrucciones `JL`/`JGE` interpretan el resultado como número con signo, mientras que `JB`/`JAE` lo interpretan sin signo. En el juego, la fila y la columna de la cabeza se mantienen en registros de 8 bits. Cuando la cabeza está en la fila 0 y se mueve hacia arriba, `DEC` deja el valor `255` (`0FFh`). Con interpretación con signo, `0FFh` equivale a -1 y `JGE` lo trataría como menor que el límite del tablero, de modo que no detectaría la pared. Con interpretación sin signo, `255` es mayor que cualquier límite válido y `JAE` detecta correctamente la salida del tablero. Por eso la comparación sin signo es obligatoria en este programa.

2.3. Modo texto 80x25 y servicios BIOS (INT 10h)

En modo texto de 80 columnas por 25 filas, la memoria de video comienza en la dirección `B800:0000`; cada celda ocupa dos bytes (carácter y atributo de color). Escribir directamente en esa memoria sería la opción más rápida. Este prototipo emplea en cambio la interrupción `INT 10h` del BIOS, que es más sencilla de explicar y de verificar: es una decisión didáctica, no óptima. Los servicios utilizados se resumen en el Anexo C (Tabla 12). Los atributos de color usados son `07h` (gris, borde), `0Ah` (verde, serpiente) y `0Ch` (rojo, comida).

2.4. Lectura de teclado y temporización

La función `AH=01h` de `INT 16h` consulta el búfer de teclado *sin bloquear*: indica mediante `ZF` si hay una tecla pendiente. La función `AH=00h` extrae la tecla del búfer; para teclas extendidas, como las flechas, devuelve `AL=0` y el código de exploración (*scancode*) en `AH`.

`INIT_GAME` usa `INT 1Ah AH=00h` para obtener ticks del BIOS y mezclar una semilla para el generador de comida. `DELAY` usa `INT 15h AH=86h`, con `CX=0001h` y `DX=86A0h`, para solicitar una espera de 100 000 microsegundos (100 ms nominales); no se presenta como tiempo medido. El comportamiento de esta función en EMU8086 no fue verificado.

2.5. Cola circular y direccionamiento indirecto

Una *cola circular* es un arreglo de tamaño fijo con dos índices: uno de escritura (`head`) y otro de lectura (`tail`). Al llegar al final del arreglo, el índice vuelve a cero. En el juego, la cola almacena los índices de las celdas ocupadas por la serpiente, de la cola (más antigua) a la cabeza (más reciente). Mover la serpiente consiste en insertar una celda en `head` y, si no comió, retirar la celda de `tail`. Con un arreglo lineal habría que desplazar todos los elementos tras cada movimiento (`memmove`); la cola circular lo evita y mantiene cada operación en tiempo constante.

El acceso al arreglo `queue`, de palabras de 16 bits, usa direccionamiento indirecto por registro (`[SI]`), con el índice multiplicado por dos mediante `SHL SI,1`. El acceso al arreglo `board`, de bytes, usa `[BX]`.

3. Diseño del juego

3.1. Reglas y controles

El jugador controla una serpiente que se mueve sin detenerse. Las reglas son las siguientes:

- La serpiente comienza con tres segmentos en la fila 5, columnas 19, 20 y 21 del tablero, con la cabeza en la columna 21 y orientada a la derecha.
- Al ocupar la celda de la comida, la serpiente crece un segmento y el puntaje aumenta en uno; aparece una nueva comida en una celda libre.
- El juego termina al chocar con una pared o con el propio cuerpo. La celda que ocupa la cola en ese instante *cuenta como colisión*.
- El giro de 180° está prohibido: se rechaza la tecla cuya dirección sea opuesta a la actual.
- La tecla `ESC` termina el programa en cualquier momento y restaura el modo de video y el cursor.

La Tabla 3 resume los controles. La dirección se codifica con `dir`: 0 arriba, 1 derecha, 2 abajo, 3 izquierda.

Cuadro 3: Teclas aceptadas, códigos y dirección resultante.

Tecla	ASCII (`AL`)	Scancode (`AH`)	Dirección	`dir`
Flecha arriba	`0`	`48h`	Arriba	0
Flecha derecha	`0`	`4Dh`	Derecha	1
Flecha abajo	`0`	`50h`	Abajo	2
Flecha izquierda	`0`	`4Bh`	Izquierda	3
`W` / `w`	`57h` / `77h`	—	Arriba	0
`D` / `d`	`44h` / `64h`	—	Derecha	1
`S` / `s`	`53h` / `73h`	—	Abajo	2
`A` / `a`	`41h` / `61h`	—	Izquierda	3
`ESC`	`1Bh` (27)	—	Salir	—

3.2. Representación del tablero

El tablero lógico tiene 40 columnas por 10 filas (400 celdas) y se guarda en el arreglo `board`, con un byte por celda: `0` libre, `1` serpiente y `2` comida. El índice de una celda es fila $\times$ 40 + columna. La celda de fila r y columna c se dibuja en la pantalla en la fila $r + 2$ y la columna $c + 20$ del modo texto. Por ejemplo, la celda de la fila 5 y la columna 19 tiene índice $5 \cdot 40 + 19 = 219$. El borde se dibuja en gris (`07h`) alrededor del tablero; el puntaje aparece en la fila 0, columna 19, con el formato “Puntaje: ” seguido del número.

3.3. Estructuras de datos

La Tabla 4 lista las constantes y variables del programa. Su contenido coincide con el código fuente y se repite como Anexo B.

3.4. Procedimientos y sus contratos

El programa define exactamente diez procedimientos: `INIT_GAME`, `DRAW_BORDER`, `READ_KEY`, `MOVE_SNAKE`, `CHECK_COLLISION`, `SPAWN_FOOD`, `DRAW_CELL`, `PRINT_NUM`, `DELAY` y `GAME_OVER`. La Tabla 5 presenta sus contratos; las salidas intencionales de cada rutina se indican por separado de los registros preservados. No se emplean `PUSHA` ni `POPA`.

Cuadro 4: Constantes y variables del programa.

Nombre	Tipo	Tamaño	Uso
`BOARD_W`	`EQU`	—	Ancho del tablero: 40.
`BOARD_H`	`EQU`	—	Alto del tablero: 10.
`BOARD_SIZE`	`EQU`	—	Total de celdas: 400.
`board`	`DB`	400 bytes	Estado de cada celda (0, 1, 2).
`queue`	`DW`	400 palabras	Cola circular de índices de la serpiente.
`head`	`DW`	1 palabra	Índice de escritura de la cola (cabeza).
`tail`	`DW`	1 palabra	Índice de lectura de la cola (cola).
`dir`	`DB`	1 byte	Dirección actual (0 a 3).
`headRow`	`DB`	1 byte	Fila de la cabeza.
`headCol`	`DB`	1 byte	Columna de la cabeza.
`score`	`DW`	1 palabra	Puntaje.
`seed`	`DW`	1 palabra	Semilla del generador pseudoaleatorio, inicializada a partir de los ticks del BIOS.
`msgScore`	`DB`	cadena	Texto “Puntaje: ”.
`msgGameOver`	`DB`	cadena	Texto “GAME OVER!”.

Cuadro 5: Fichas de los procedimientos.

Procedimiento	Entradas	Salidas	Resumen
`INIT_GAME`	Ninguna (usa constantes).	Variables y `board` inicializados.	Limpia el tablero, coloca los tres segmentos iniciales, fija `dir`, `headRow`, `headCol`, `head`, `tail` y puntaje.
`DRAW_BORDER`	Ninguna.	Borde dibujado.	Dibuja el marco gris del tablero mediante bucles con `LOOP`.
`READ_KEY`	Estado del búfer de teclado.	`AL`: 0 si el juego continúa (ver Sección 5.1). Actualiza `dir`.	Consulta sin bloquear, consume la tecla, detecta `ESC`, flechas y WASD, y rechaza el giro de 180°.
`MOVE_SNAKE`	`dir`, `headRow`, `headCol`, `head`, `tail`.	`AL=1` si hubo colisión; actualiza cola y tablero.	Calcula la nueva cabeza, consulta la colisión, encola la nueva celda y, si no come, libera la cola.
`CHECK_COLLISION`	Fila y columna candidatas.	`AL=1` si choca con pared o cuerpo.	Compara con límites mediante `JAE`, calcula el índice con `MUL` y consulta `board`.
`SPAWN_FOOD`	`seed`, `board`.	Comida en una celda libre (o ninguna, tras 1000 intentos).	Genera posiciones pseudoaleatorias hasta hallar una celda libre.
`DRAW_CELL`	Índice de celda, carácter y atributo.	Celda dibujada.	Convierte índice a fila y columna con `DIV`, posiciona el cursor y escribe el carácter con `INT 10h`.
`PRINT_NUM`	Número (puntaje).	Número impreso.	Descompone el número en dígitos con `DIV` y los imprime con `INT 21h`.
`DELAY`	Ninguna.	Ninguna; preserva `AX`, `CX` y `DX`.	Solicita una espera nominal de 100 ms con `INT 15h AH=86h`, `CX=0001h` y `DX=86A0h`.

Procedimiento	Entradas	Salidas	Resumen
`GAME_OVER`	`score`.	Mensaje y puntaje final.	Muestra "GAME OVER!" y el puntaje en la fila 14 y espera una tecla.

4. Desarrollo e implementación

4.1. Altas y bajas del diseño

El diseño evolucionó mediante decisiones de incorporación (altas) y de descarte (bajas). Esta subsección las narra y justifica.

Altas. *Cola circular.* Se adoptó una cola circular de índices para representar la serpiente. La alternativa, un arreglo lineal con desplazamiento de elementos en cada paso, habría exigido copiar hasta 400 palabras por movimiento. La cola circular reduce el costo a una inserción y, como máximo, una extracción.

Separación entre `CHECK_COLLISION` y el manejo de comida. La verificación de choque solo informa si la celda destino es mortal; la decisión de comer, crecer o liberar la cola corresponde a `MOVE_SNAKE`. Esta separación mantiene cada procedimiento con una responsabilidad única y facilita la explicación en clase.

Dibujo selectivo. En lugar de redibujar el tablero completo en cada fotograma, el programa actualiza únicamente la nueva cabeza, la celda de la cola cuando esta se libera y la comida al generarse.

Bajas. *Redibujado completo del tablero.* Se descartó porque provocaba parpadeo visible y un costo innecesario: cada fotograma habría emitido cientos de llamadas a `INT 10h`.

Tabla precalculada de fila $\times 40$. Se descartó porque añadía complejidad de datos y de acceso sin aportar una mejora relevante en un prototipo cuyo cuello de botella es el retardo deliberado. El índice se calcula con `MUL`, lo que además permite ejercitar la Práctica 5.

Sonido y pausa. Quedaron fuera del alcance: no forman parte de los objetivos del curso y habrían incrementado el tamaño y la superficie de pruebas del programa.

Corrección al comer. Se corrigió la conservación de `BX` (índice de la nueva cabeza) y `DX` (coordenadas) alrededor de la llamada a `INT 10h AH=02h` que posiciona el cursor del puntaje. Antes, los valores de `BH`, `DH` y `DL` sobrescribían esos registros: la cabeza lógica saltaba a (0,28), la comida podía permanecer dibujada y generarse una segunda semilla, y la serpiente reaparecía desplazada. La corrección guarda ambos registros con `PUSH` antes de la llamada y los restaura con `POP` después de `PRINT_NUM`, antes de `SPAWN_FOOD`.

4.2. Decisiones de implementación destacadas

- **Comparación sin signo.** La detección de paredes usa `CMP` con `JAE`, porque `DEC` sobre una fila o columna en cero produce `255` (véase el Marco teórico).
- **Giro de** $180°$**.** `READ_KEY` rechaza la nueva dirección si es igual a (`dir` $+ 2$) `AND` 3, expresión que obtiene la dirección opuesta en aritmética módulo 4.
- **Generador pseudoaleatorio.** `SPAWN_FOOD` aplica la recurrencia `seed` $\leftarrow$ `seed` $\cdot\, 25173 + 13849$ con `MUL` de 16 bits y obtiene la posición como `seed` mód 400 con `DIV` de 16 bits, con `DX=0` previo.
- **Límite de intentos.** `SPAWN_FOOD` realiza como máximo 1000 intentos, de modo que no se cuelga si el tablero está lleno.
- **Salida limpia.** Al salir, el programa restaura el modo de video y el cursor mediante `INT 10h`, `AH=01h`, `CX=0607h`.
- **Preservación de registros al comer.** `MOVE_SNAKE` preserva `BX` y `DX` alrededor del posicionamiento e impresión del puntaje; esto aplica `PUSH`/`POP` para impedir que los parámetros de cursor destruyan el índice y las coordenadas del movimiento.

4.3. Tabla de rastreo: Práctica del curso a código

La Tabla 6 relaciona cada práctica con las instrucciones que realmente aparecen en el código. No se incluyen instrucciones ausentes del programa (por ejemplo, no se emplea `IMUL`). Las banderas que deciden los saltos están en la Tabla 2.

Cuadro 6: Rastreo entre las prácticas del curso y el código de `snakeGame.asm`.

Práctica	Instrucción / tema	Procedimiento real del código	Procedimiento del `.asm` donde aparece
2	`ADD/ADC`	Suma del incremento del generador con propagación de acarreo.	`SPAWN_FOOD`
2	`SUB`	Inicialización de valores; pone a cero el registro usado para limpiar el tablero.	`INIT_GAME`
2	`XOR`	Puesta a cero de registros.	`INIT_GAME`, `READ_KEY`, `CHECK_COLLISION`, `PRINT_NUM`, `SPAWN_FOOD`
2	`AND 3`	Módulo 4 de la dirección opuesta.	`READ_KEY`
2	`SHL SI,1`	Conversión de índice a desplazamiento en palabras.	`MOVE_SNAKE`
3	`MOV` inmediato	Carga de constantes y coordenadas iniciales.	`INIT_GAME`, `DRAW_BORDER`
3	`MOV` registro–registro	Preparación de parámetros de pantalla.	`DRAW_CELL`
3	`MOV` directo a memoria	Actualización de variables.	`MOVE_SNAKE`, `INIT_GAME`
3	`MOV` indirecto	`[BX]` sobre `board`; `[SI]` sobre `queue`.	`CHECK_COLLISION` (`[BX]`); `MOVE_SNAKE`, `SPAWN_FOOD` (`[SI]`)
3	`org 100h`; `XCHG BX,DI`	Origen del formato COM; intercambio de registros.	Inicio del programa; `MOVE_SNAKE`
4	`CMP`, `JE`, `JNE`, `JB`, `JAE`	Decisiones de teclas, límites y estados de celda.	Todo el código
4	`JMP`	Saltos incondicionales y repetición del bucle.	`MOVE_SNAKE`, `main_loop`
4	`LOOP`	Bucles con contador.	`DRAW_BORDER`, `SPAWN_FOOD`, `PRINT_NUM`
4	`CALL/RET`	Invocación de procedimientos.	Todos los procedimientos
5	`MUL` de 8 bits	Cálculo de fila $\times 40 +$ columna.	`CHECK_COLLISION`
5	`MUL` de 16 bits	Multiplicación de la semilla.	`SPAWN_FOOD`
5	`DIV` de 8 bits	Conversión de índice a fila y columna.	`DRAW_CELL`
5	`DIV` de 16 bits	Dígitos decimales; resto módulo 400.	`PRINT_NUM`, `SPAWN_FOOD`
5	`INC/DEC`	Desplazamiento de la cabeza.	`MOVE_SNAKE`
5	`PUSH/POP`	Preservación de registros; en la rama de comida se guardan `BX` y `DX` alrededor de la actualización del puntaje.	Todos los procedimientos; `MOVE_SNAKE` (rama `ms_eat`)
5	`EQU`, `DB`, `DW`	Constantes y datos.	Segmento de datos

5. Funcionamiento

5.1. Bucle principal

El programa inicializa el juego, dibuja el borde, genera la primera comida y entra en el bucle principal. En cada iteración ejecuta, en este orden: `READ_KEY`, `MOVE_SNAKE`, verificación de `AL=1` (choque) y `DELAY`; luego repite. Leer la tecla antes de mover garantiza que el movimiento use la dirección más reciente. `DELAY` solicita al BIOS una espera nominal de 100 ms al final de cada ciclo.

```
INIT_GAME
DRAW_BORDER
SPAWN_FOOD
repetir:
    READ_KEY
    si READ_KEY indica salir: terminar
    MOVE_SNAKE
    si MOVE_SNAKE devolvio 1 (choque):
        GAME_OVER
        terminar
    DELAY
    volver a repetir
```

Listing 1: Bucle principal (pseudocódigo).

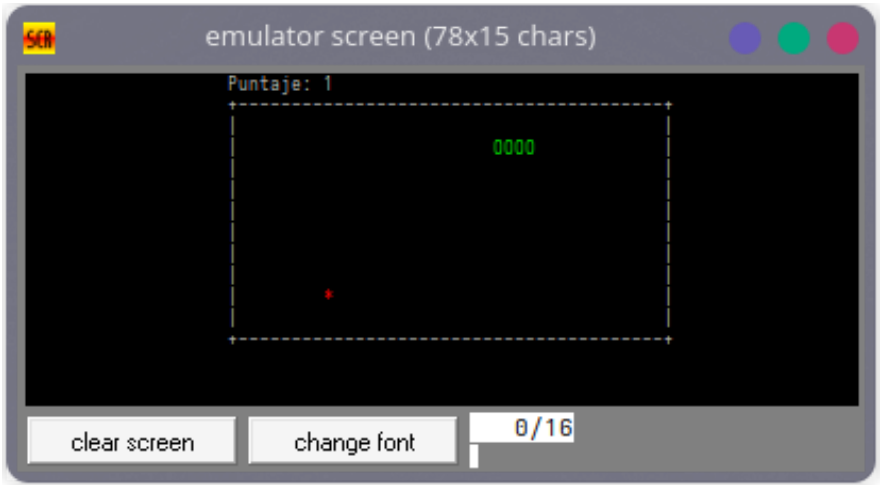


Figura 1: Tablero y elementos durante la partida

5.2. Flujo detallado de una tecla

La figura de texto del Listado 2 describe el recorrido desde que el usuario presiona `W` hasta que `READ_KEY` retorna 0 con la dirección actualizada.

```
Usuario presiona W
        |
        v
INT 16h AH=01h  (consulta no bloqueante)
        |
   hay tecla? --no--> retorna 0 (sin cambio de dir)
        |si
        v
INT 16h AH=00h  (consume la tecla)
        |
```

```
AL = 27 (ESC)? --si--> senal de salida
     |no
     v
AL = 0 (tecla extendida)?
  |si                          |no
  v                             v
AH = 48h/50h/4Bh/4Dh?       AL = W/S/A/D (may. o min.)?
  |si          |no             |si           |no
  v            v               v             v
nueva dir    ignorar        nueva dir      ignorar
     \---------------  -----------------/
                     \/
     nueva == (dir+2) AND 3 ?   (giro de 180)
         |si                      |no
         v                        v
     rechazar              dir = nueva
         \--------  --------/
                  \/
            retorna 0
```

Listing 2: Flujo de una tecla en READ_KEY.

La combinación de la Tabla 3 y del Listado 2 define el comportamiento completo del teclado.

5.3. Flujo detallado de un movimiento

`MOVE_SNAKE` calcula la nueva celda de la cabeza según `dir`: incrementa o decrementa `headRow` o `headCol`. Llama a `CHECK_COLLISION`; si hay choque, retorna `AL=1`. En caso contrario consulta `board` en la nueva celda y se bifurca:

- **No come** (celda libre): encola el índice nuevo, marca `board=1`, dibuja la nueva cabeza, libera la celda de `tail` (`board=0`), la borra de pantalla y avanza `tail`.
- **Come** (`board=2`): se desvía a `ms_eat`; se detalla más adelante.

Micro-traza: presiono `W` yendo a la derecha. La traza describe estados y orden de ejecución, no duraciones medidas. Las celdas iniciales son $219 = 5 \cdot 40 + 19$, $220 = 5 \cdot 40 + 20$ y $221 = 5 \cdot 40 + 21$; la nueva cabeza será $181 = 4 \cdot 40 + 21$.

- **Antes de la espera.** El usuario presiona `W`. Estado: `dir`=1, `headRow`=5, `headCol`=21, `queue`=[219, 220, 221], `head`=3, `tail`=0, `board[219]`=1, `board[181]`=0, `score`=0. La tecla permanece en el búfer del BIOS.
- **Durante `DELAY`.** La espera explícita sigue activa; no se atribuye una duración medida y el estado del juego no cambia.
- **En la iteración siguiente.** `READ_KEY` consume `W`; la nueva dirección es 0 y $(1 + 2)$ `AND` $3 = 3 \neq 0$, de modo que no es giro de 180° y `dir` pasa de 1 a 0.
- **Después de `MOVE_SNAKE`.** `headRow` pasa de 5 a 4; `headCol` permanece en 21; el índice nuevo es $181 = 4 \cdot 40 + 21$. La cola lógica pasa de [219, 220, 221] a [220, 221, 181]; `head` pasa de 3 a 4 y `tail` de 0 a 1. Además, `board[219]`=0, `board[181]`=1 y `score`=0.

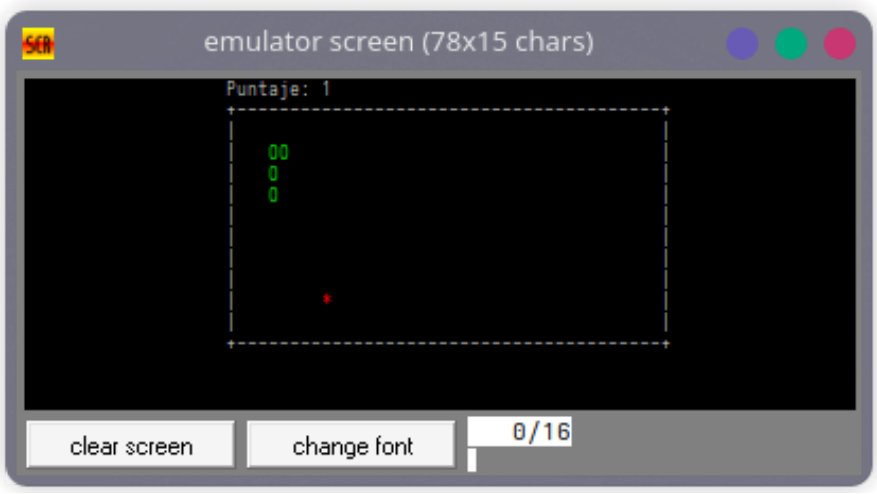


Figura 2: Movimiento de la serpiente tras cambiar de dirección

5.4. Flujo detallado de "comer"

Cuando `board[idx]` vale 2, `MOVE_SNAKE` se desvía a la etiqueta `ms_eat`. La rama incrementa `score`, conserva `BX` y `DX` con `PUSH` antes de llamar a `INT 10h AH=02h` para posicionar el cursor del puntaje, invoca `PRINT_NUM` y restaura esos registros con `POP` antes de `SPAWN_FOOD`. Así evita que `BH=0`, `DH=0` y `DL=28` alteren el índice de la nueva cabeza o la fila y columna candidatas. Luego genera la siguiente comida, encola `idx` como nueva cabeza y actualiza esa celda como serpiente; **no se avanza** `tail`, por lo que la serpiente crece un segmento.

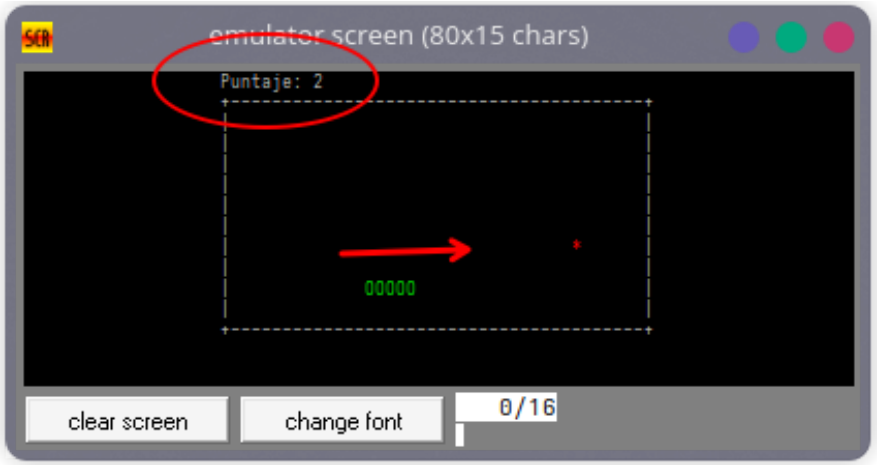


Figura 3: Consumo de una semilla y crecimiento

5.5. Flujo detallado de un choque

Considérese la cabeza en `headRow`=0 con `dir`=0 (arriba). `DEC DH` convierte la fila candidata en `255`. `CHECK_COLLISION` compara con `BOARD_H`=10 y, al ser `JAE` una comparación sin signo, $255 \geq 10$ se cumple; la rutina devuelve `AL=1`. `MOVE_SNAKE` propaga ese valor y el bucle principal invoca `GAME_OVER`. Con `JGE`, `255` se interpretaría como -1 y no se detectaría la pared; por eso se usa `JAE`.

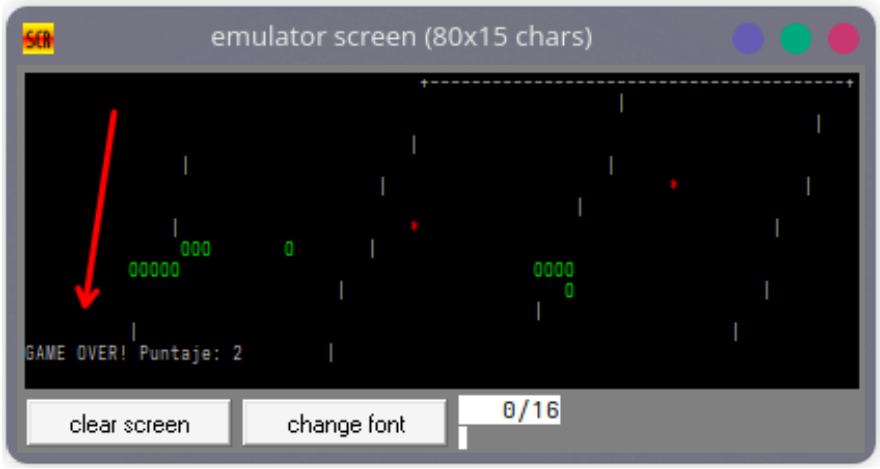


Figura 4: Choque con una pared y fin de partida

6. Pseudocódigo

Los siguientes listados describen cada procedimiento en lenguaje de alto nivel. Los nombres de procedimientos van en mayúsculas y las variables en minúsculas. Los listados no incluyen direcciones ni registros.

6.1. Bucle principal

Véase el Listado 1.

6.2. INIT_GAME

```
INIT_GAME:
    poner en libre todas las celdas de board
    head = 0
    tail = 0
    dir = derecha
    score = 0
    headRow = 5
    para col desde 19 hasta 21:
        marcar board[5*40 + col] como serpiente
        encolar el indice en queue
        dibujar la celda de serpiente
    headCol = 21
```

Listing 3: Pseudocódigo de INIT_GAME.

6.3. DRAW_BORDER

```
DRAW_BORDER:
    para cada columna del marco:
        dibujar borde superior e inferior
    para cada fila del marco:
        dibujar borde izquierdo y derecho
```

Listing 4: Pseudocódigo de DRAW_BORDER.

6.4. READ_KEY

```
READ_KEY:
    si no hay tecla en el buffer:
        retornar 0
    leer y consumir la tecla
    si es ESC:
        indicar salida
    si es flecha o WASD (mayuscula o minuscula):
        nueva = direccion correspondiente
        si nueva es la opuesta de dir:
            ignorar
        si no:
            dir = nueva
    retornar 0
```

Listing 5: Pseudocódigo de READ_KEY.

6.5. CHECK_COLLISION

```
CHECK_COLLISION:
    si fila no es menor que alto del tablero:
        retornar 1
    si columna no es menor que ancho del tablero:
        retornar 1
    idx = fila * 40 + columna
    si board[idx] es serpiente:
        retornar 1
    retornar 0
```

Listing 6: Pseudocódigo de CHECK_COLLISION.

6.6. MOVE_SNAKE

```
MOVE_SNAKE:
    calcular nueva fila y nueva columna segun dir
    si CHECK_COLLISION indica choque:
        retornar 1
    idx = nueva fila * 40 + nueva columna
    si board[idx] es comida:
        score = score + 1
        guardar BX y DX con PUSH
        posicionar el cursor del puntaje con INT 10h AH=02h
        PRINT_NUM
        restaurar DX y BX con POP
        SPAWN_FOOD
        encolar idx
        board[idx] = serpiente
        dibujar nueva cabeza
    si no:
        encolar idx
        board[idx] = serpiente
        dibujar nueva cabeza
        antigua = elemento en tail
        board[antigua] = libre
        borrar la celda antigua en pantalla
        avanzar tail circularmente
```

```
    actualizar headRow y headCol
    retornar 0
```

Listing 7: Pseudocódigo de MOVE_SNAKE.

6.7. SPAWN_FOOD

```
SPAWN_FOOD:
    repetir hasta 1000 intentos:
        seed = seed * 25173 + 13849
        pos = seed modulo 400
        si board[pos] es libre:
            board[pos] = comida
            dibujar comida
            retornar
    retornar sin generar comida
```

Listing 8: Pseudocódigo de SPAWN_FOOD.

6.8. DRAW_CELL

```
DRAW_CELL:
    fila = idx dividido 40
    columna = resto de idx dividido 40
    posicionar cursor en (fila + 2, columna + 20)
    escribir caracter con su color
```

Listing 9: Pseudocódigo de DRAW_CELL.

6.9. PRINT_NUM

```
PRINT_NUM:
    posicionar cursor tras la etiqueta de puntaje
    obtener los digitos de score dividiendo entre 10
    imprimir los digitos de izquierda a derecha
    imprimir espacios de relleno
```

Listing 10: Pseudocódigo de PRINT_NUM.

6.10. DELAY

```
DELAY:
    preservar AX, CX y DX
    AH = 86h
    CX = 0001h
    DX = 86A0h
    llamar INT 15h
    restaurar DX, CX y AX
```

Listing 11: Pseudocódigo de DELAY.

6.11. GAME_OVER

```
GAME_OVER:
    mostrar GAME OVER! en la fila 14
    mostrar el puntaje final
    esperar una tecla
```

Listing 12: Pseudocódigo de GAME_OVER.

6.12. Pseudocódigo del programa completo

```
programa Snake:
    INIT_GAME
    DRAW_BORDER
    SPAWN_FOOD
    mientras verdadero:
        READ_KEY
        si hubo ESC:
            restaurar video y cursor
            terminar
        MOVE_SNAKE
        si hubo choque:
            GAME_OVER
            restaurar video y cursor
            terminar
        DELAY
```

Listing 13: Pseudocódigo del programa completo.

7. Análisis de rendimiento

7.1. ¿Por qué el juego es "lento"?

El programa no es lento por naturaleza: por fotograma ejecuta pocas instrucciones y alrededor de cuatro llamadas a `INT 10h`. El ritmo lo determina la espera explícita de `DELAY`, que solicita 100 ms nominales mediante `INT 15h AH=86h` con `CX=0001h` y `DX=86A0h`. Ese valor es una solicitud BIOS, no una medición; la velocidad percibida varió entre ejecuciones del mismo código según la velocidad de emulación. El comportamiento de `INT 15h AH=86h` en EMU8086 no fue verificado.

7.2. Costo por fotograma

La Tabla 7 resume los factores cualitativos. No se asignan ciclos ni duraciones de ejecución que no hayan sido medidos.

7.3. Cuellos de botella y latencia de entrada

La espera explícita controla el ritmo; no se ha medido el costo de las demás operaciones. La velocidad percibida varió entre ejecuciones del mismo código en EMU8086, dependiendo de la velocidad de emulación. Además, `READ_KEY` lee una sola tecla por ciclo y no vacía el búfer: mantener una tecla presionada puede acumular pulsaciones en el búfer del BIOS, por lo que la serpiente responde con retraso. Es una limitación conocida y no corregida.

Cuadro 7: Factores de costo por fotograma.

Factor	Costo por frame	¿Cuello de botella?
`DELAY`	Espera solicitada nominalmente de 100 ms con `INT 15h AH=86h`.	Sí, determina el ritmo.
Instrucciones del juego	Pocas por fotograma; sin estimación de ciclos.	No se ha medido.
`INT 10h`	Alrededor de cuatro llamadas por fotograma.	No se ha medido como cuello de botella.
`INT 16h`	Consulta el teclado en `READ_KEY`.	No se ha medido.
`INT 1Ah`	Consulta de ticks durante `INIT_GAME` para inicializar `seed`.	No; inicializa el generador.
`INT 15h`	Una solicitud de espera por fotograma mediante `AH=86h`.	Sí; controla el ritmo.
`INT 21h`	Interviene al imprimir el puntaje cuando se come.	No se ha medido.

8. Limitaciones y consideraciones

8.1. Limitaciones funcionales del juego

Cuadro 8: Limitaciones funcionales.

Limitación	Descripción
Tablero lleno sin victoria	No existe condición de victoria cuando la serpiente ocupa todo el tablero.
Celda de la cola	La celda de la cola cuenta como colisión, aunque en ese instante quedaría libre.
Sin reinicio	Tras `GAME_OVER` no se ofrece jugar de nuevo.
Sin récord	No se conserva la mejor puntuación.
Ritmo del juego	Usa una solicitud fija nominal de 100 ms mediante `DELAY`; el tiempo percibido depende de la velocidad de emulación.
Sin pausa	No hay tecla de pausa.
Sin sonido	No se emite ningún sonido.
Semilla pseudoaleatoria	Se inicializa a partir de los ticks del BIOS; si dos ejecuciones obtienen los mismos ticks iniciales, pueden producir la misma secuencia de comida.

8.2. Limitaciones técnicas del código

Cuadro 9: Limitaciones técnicas.

Limitación	Descripción
`SPAWN_FOOD` falla en silencio	Tras 1000 intentos sin celda libre, retorna sin comida y sin avisar.
`DRAW_CELL`	Asume un índice de celda ≤ 399; no valida el rango.

Limitación	Descripción
`CHECK_COLLISION`	Es frágil ante un cambio de `JAE` a `JGE`, que dejaría de detectar las paredes.
`queue` no se reinicia	Los valores antiguos permanecen; solo `head` y `tail` definen el contenido válido.
`READ_KEY`	Lee una sola tecla por ciclo y no vacía el búfer; mantener una tecla presionada puede acumular pulsaciones y retrasar la respuesta. También ignora teclas no previstas.
EMU8086 y temporización	La velocidad percibida varió entre ejecuciones del mismo código según la velocidad de emulación; no se registran mediciones. El comportamiento de `INT 15h AH=86h` no está verificado y 100 ms es solo un valor nominal, no medido.
`PRINT_NUM`	Deja tres espacios tras el número impreso.

8.3. Limitaciones de portabilidad y entorno

Cuadro 10: Limitaciones de portabilidad.

Aspecto	Descripción
Plataforma	Diseñado para EMU8086 y DOS; no se garantiza en otros entornos.
Formato COM	Tamaño máximo de 64 KB.
EMU8086 y temporización	La velocidad percibida varió entre ejecuciones del mismo código según la velocidad de emulación. El comportamiento de `INT 15h AH=86h` en EMU8086 no fue verificado; 100 ms es el intervalo nominal solicitado, no medido.
Latencia de teclado	`READ_KEY` procesa una tecla por ciclo y no vacía el búfer; mantener una tecla presionada acumula pulsaciones y retrasa la respuesta. No está corregido.

8.4. Posibles mejoras futuras

Entre las mejoras posibles, que *no* forman parte del programa actual, se cuentan: velocidad progresiva, reinicio con la tecla `R`, récord en disco, cabeza diferenciada del cuerpo, sonido con `INT 10h AH=0Eh`, pausa con la tecla `P` y tablero configurable mediante `EQU`.

9. Pruebas y validación

9.1. Procedimiento de prueba recomendado

1. Abrir `snakeGame.asm` en EMU8086 y ensamblarlo sin errores.
2. Ejecutar el programa en el emulador y comprobar que se dibuja el borde, la serpiente inicial y la comida.
3. Jugar durante unos 30 segundos probando flechas y WASD.
4. Verificar el rechazo del giro de 180°.
5. Provocar choques con cada una de las cuatro paredes y con el cuerpo.
6. Comer varias veces y observar el crecimiento y el puntaje con una, dos y tres cifras.

7. Colocar una semilla en una fila entre 7 y 9 y comerla; comprobar que desaparece, no se duplica y que la cabeza continúa desde la celda comida.
8. Verificar `GAME_OVER` y la espera de tecla.
9. Repetir y salir con `ESC`; confirmar que el cursor y el modo de video se restauran.
10. Marcar cada ítem de la lista de verificación.

10. Anexos

10.1. Código fuente completo (snakeGame.asm)

El Listado 14 incluye el código fuente completo.

```
org 100h
jmp inicio

; PROCEDIMIENTOS
; ---------------------------------------------------------------
; INIT_GAME: Inicializa video, board, serpiente, puntaje y comida
; Entrada: ninguna
; Salida: ninguna
; Registros modificados: todos (se preservan con PUSH/POP)
; ---------------------------------------------------------------
INIT_GAME PROC
    push ax
    push bx
    push cx
    push dx
    push si
    push di
    ; Modo texto 80x25 (PRACTICA 3: MOV inmediato)
    mov ax, 0003h
    int 10h
    ; Ocultar cursor
    mov ah, 01h
    mov cx, 2000h
    int 10h
    ; Limpiar board (400 bytes a 0)
    mov di, offset board
    mov cx, BOARD_SIZE
    sub al, al                  ; PRACTICA 5: SUB, AL=0
clear_b:
    mov [di], al
    inc di
    loop clear_b
    ; Inicializar cola: 3 segmentos en fila 5, cols 19,20,21
    ; Indices: 5*40+19=219, 220, 221
    mov word ptr [queue], 219
    mov word ptr [queue+2], 220
    mov word ptr [queue+4], 221
    mov head, 3                 ; proximo indice libre
    mov tail, 0
    mov byte ptr dir, 1         ; derecha
    mov byte ptr headRow, 5
    mov byte ptr headCol, 21
    mov score, 0
    ; Marcar en board (PRACTICA 3: MOV directo con [BX/indice])
```

```
    mov byte ptr board[219], 1
    mov byte ptr board[220], 1
    mov byte ptr board[221], 1
    ; Semilla desde reloj BIOS
    mov ah, 00h
    int 1Ah
    mov seed, dx
    xor seed, cx                ; PRACTICA 2: XOR para mezclar
    ; Dibujar borde
    call DRAW_BORDER
    ; Dibujar serpiente inicial
    mov bx, 219
    mov al, 'O'
    mov ah, 0Ah
    call DRAW_CELL
    mov bx, 220
    call DRAW_CELL
    mov bx, 221
    call DRAW_CELL
    ; Imprimir el mensaje del puntaje
    mov ah, 02h
    mov bh, 0
    mov dh, 0
    mov dl, 19
    int 10h
    mov dx, offset msgScore
    mov ah, 09h
    int 21h
    ; Imprimir puntaje inicial
    mov ah, 02h
    mov bh, 0
    mov dh, 0
    mov dl, 28
    int 10h
    mov ax, 0
    call PRINT_NUM
    ; Generar primera comida
    call SPAWN_FOOD
    pop di
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
INIT_GAME ENDP

; ---------------------------------------------------------
; DRAW_BORDER: Dibuja el borde gris del tablero
; Entrada: ninguna
; Salida: ninguna
; Registros modificados: AX, BX, CX, DX (se preservan)
; ---------------------------------------------------------
DRAW_BORDER PROC
    push ax
    push bx
    push cx
    push dx
    ; Borde superior: fila 1, col 19, 42 '-'
```

```
    mov ah, 02h
    mov bh, 0
    mov dh, 1
    mov dl, 19
    int 10h
    mov ah, 09h
    mov al, '-'
    mov bh, 0
    mov bl, 07h
    mov cx, 42
    int 10h
    ; Borde inferior: fila 12, col 19, 42 '-'
    mov ah, 02h
    mov bh, 0
    mov dh, 12
    mov dl, 19
    int 10h
    mov ah, 09h
    mov al, '-'
    mov bh, 0
    mov bl, 07h
    mov cx, 42
    int 10h
    ; Bordes laterales: filas 2..11, cols 19 y 60
    mov cx, 10
    mov dh, 2
db_side:
    ; Izquierdo
    mov ah, 02h
    mov bh, 0
    mov dl, 19
    push cx
    int 10h
    mov ah, 09h
    mov al, '|'
    mov bh, 0
    mov bl, 07h
    mov cx, 1
    int 10h
    ; Derecho
    mov ah, 02h
    mov bh, 0
    mov dl, 60
    int 10h
    mov ah, 09h
    mov al, '|'
    mov bh, 0
    mov bl, 07h
    mov cx, 1
    int 10h
    pop cx
    inc dh
    loop db_side
    ; Esquinas: 4 '+'
    ; Sup izq (1,19)
    mov ah, 02h
    mov bh, 0
    mov dh, 1
    mov dl, 19
```

```
    int 10h
    mov ah, 09h
    mov al, '+'
    mov bh, 0
    mov bl, 07h
    mov cx, 1
    int 10h
    ; Sup der (1,60)
    mov ah, 02h
    mov bh, 0
    mov dh, 1
    mov dl, 60
    int 10h
    mov ah, 09h
    mov al, '+'
    mov bh, 0
    mov bl, 07h
    mov cx, 1
    int 10h
    ; Inf izq (12,19)
    mov ah, 02h
    mov bh, 0
    mov dh, 12
    mov dl, 19
    int 10h
    mov ah, 09h
    mov al, '+'
    mov bh, 0
    mov bl, 07h
    mov cx, 1
    int 10h
    ; Inf der (12,60)
    mov ah, 02h
    mov bh, 0
    mov dh, 12
    mov dl, 60
    int 10h
    mov ah, 09h
    mov al, '+'
    mov bh, 0
    mov bl, 07h
    mov cx, 1
    int 10h
    pop dx
    pop cx
    pop bx
    pop ax
    ret
DRAW_BORDER ENDP

; ------------------------------------------------------------
; READ_KEY: Lee teclado (no bloqueante). Actualiza dir o indica ESC
; Entrada: ninguna
; Salida: AL = 1 si ESC, 0 si no
; Registros modificados: AX (salida), BX (se preserva)
; ------------------------------------------------------------
READ_KEY PROC
    push bx
    ; Hay tecla? INT 16h AH=01h: ZF=1 -> no hay
```

```
222     mov ah, 01h
223     int 16h
224     jz rk_nokey
225     ; Consumir tecla
226     mov ah, 00h
227     int 16h
228     ; AL = ASCII, AH = scancode
229     cmp al, 27                  ; ESC?
230     jne rk_arrow
231     mov al, 1
232     jmp rk_done
233 rk_arrow:
234     cmp al, 0                   ; tecla extendida?
235     jne rk_wasd
236     ; Flechas: AH = scancode
237     cmp ah, 48h
238     jne rk_ad
239     mov bl, 0                   ; arriba
240     jmp rk_set
241 rk_ad:
242     cmp ah, 50h
243     jne rk_al
244     mov bl, 2                   ; abajo
245     jmp rk_set
246 rk_al:
247     cmp ah, 4Bh
248     jne rk_ar
249     mov bl, 3                   ; izquierda
250     jmp rk_set
251 rk_ar:
252     cmp ah, 4Dh
253     jne rk_nokey
254     mov bl, 1                   ; derecha
255     jmp rk_set
256 rk_wasd:
257     ; WASD (mayusculas y minusculas)
258     cmp al, 'w'
259     je rk_w
260     cmp al, 'W'
261     je rk_w
262     cmp al, 's'
263     je rk_s
264     cmp al, 'S'
265     je rk_s
266     cmp al, 'a'
267     je rk_a
268     cmp al, 'A'
269     je rk_a
270     cmp al, 'd'
271     je rk_d
272     cmp al, 'D'
273     je rk_d
274     jmp rk_nokey
275 rk_w:
276     mov bl, 0
277     jmp rk_set
278 rk_s:
279     mov bl, 2
280     jmp rk_set
```

```
rk_a:
    mov bl, 3
    jmp rk_set
rk_d:
    mov bl, 1
rk_set:
    ; Rechazar giro de 180 grados: nueva == (dir+2) AND 3
    mov al, dir
    add al, 2
    and al, 3                   ; PRACTICA 2: AND 3 acota a 0..3
    cmp bl, al
    je rk_nokey                 ; rechazar
    mov dir, bl
rk_nokey:
    xor al, al
rk_done:
    pop bx
    ret
READ_KEY ENDP

; -------------------------------------------------------------
; CHECK_COLLISION: Verifica pared o cuerpo en (DH,DL)
; Entrada: DH = fila, DL = columna
; Salida: AL = 0 libre, 1 pared, 2 cuerpo; BX = indice si no es pared
; Registros modificados: AX, BX (salidas)
; -------------------------------------------------------------
CHECK_COLLISION PROC
    ; Paredes (comparacion sin signo: 0-1 da 255/65535)
    cmp dh, BOARD_H
    jae cc_wall                 ; Salta si DH >= BOARD_H, comparacion sin signo
    cmp dl, BOARD_W
    jae cc_wall
    ; Indice = fila*40 + col
    mov al, dh
    xor ah, ah
    mov bl, 40
    mul bl                      ; PRACTICA 5: MUL 8 bits -> AX
    mov bl, dl
    xor bh, bh
    add ax, bx                  ; PRACTICA 5: ADD
    mov bx, ax                  ; BX = indice
    cmp byte ptr board[bx], 1
    je cc_body
    xor al, al
    ret
cc_wall:
    mov al, 1
    ret
cc_body:
    mov al, 2
    ret
CHECK_COLLISION ENDP

; -------------------------------------------------------------
; MOVE_SNAKE: Mueve la serpiente un paso
; Entrada: ninguna (usa dir, headRow, headCol)
; Salida: AL = 0 OK, 1 game over
; Registros modificados: AX (salida), BX, DX, SI, DI (se preservan)
; -------------------------------------------------------------
```

```
340 MOVE_SNAKE PROC
341     push bx
342     push dx
343     push si
344     push di
345     ; Nueva posicion segun dir
346     mov dh, headRow
347     mov dl, headCol
348     mov al, dir
349     cmp al, 0
350     jne ms_nu
351     dec dh                      ; arriba
352     jmp ms_chk
353 ms_nu:
354     cmp al, 1
355     jne ms_nr
356     inc dl                      ; derecha
357     jmp ms_chk
358 ms_nr:
359     cmp al, 2
360     jne ms_nl
361     inc dh                      ; abajo
362     jmp ms_chk
363 ms_nl:
364     dec dl                      ; izquierda
365 ms_chk:
366     call CHECK_COLLISION
367     cmp al, 0
368     je ms_ok
369     mov al, 1                   ; colision -> game over
370     jmp ms_done
371 ms_ok:
372     ; BX = indice nueva cabeza
373     cmp byte ptr board[bx], 2
374     je ms_eat
375     ; --- Movimiento normal: borrar cola ---
376     mov si, tail
377     shl si, 1                   ; PRACTICA 2: SHL para indice de palabra
378     mov di, queue[si]           ; DI = indice de la cola
379     mov byte ptr board[di], 0
380     ; Dibujar espacio (PRACTICA 3: XCHG natural)
381     xchg bx, di                 ; BX=cola, DI=nueva cabeza
382     mov al, ' '
383     mov ah, 07h
384     call DRAW_CELL
385     xchg bx, di                 ; BX=nueva cabeza, DI=cola
386     ; Avanzar tail
387     inc tail
388     cmp tail, BOARD_SIZE
389     jb ms_t_ok
390     mov tail, 0
391 ms_t_ok:
392     jmp ms_enq
393 ms_eat:
394     ; --- Comer: crecer ---
395     inc score
396     push bx                     ; guardar indice de la nueva cabeza
397     push dx                     ; guardar fila/columna de la nueva cabeza
398     ; Mostrar puntaje
```

```
399     mov ah, 02h
400     mov bh, 0
401     mov dh, 0
402     mov dl, 28
403     int 10h
404     mov ax, score
405     call PRINT_NUM
406     pop dx
407     pop bx
408     ; Nueva comida
409     call SPAWN_FOOD
410 ms_enq:
411     ; Encolar nueva cabeza
412     mov si, head
413     shl si, 1
414     mov queue[si], bx
415     mov byte ptr board[bx], 1
416     ; Avanzar head
417     inc head
418     cmp head, BOARD_SIZE
419     jb ms_h_ok
420     mov head, 0
421 ms_h_ok:
422     ; Actualizar posicion de la cabeza
423     mov headRow, dh
424     mov headCol, dl
425     ; Dibujar nueva cabeza
426     mov al, 'O'
427     mov ah, 0Ah
428     call DRAW_CELL
429     xor al, al
430 ms_done:
431     pop di
432     pop si
433     pop dx
434     pop bx
435     ret
436 MOVE_SNAKE ENDP
437
438 ; ---------------------------------------------------------
439 ; SPAWN_FOOD: Genera comida en celda libre aleatoria
440 ; Entrada: ninguna (usa seed)
441 ; Salida: coloca comida en board y la dibuja
442 ; Registros modificados: AX, BX, CX, DX, SI (se preservan)
443 ; ---------------------------------------------------------
444 SPAWN_FOOD PROC
445     push ax
446     push bx
447     push cx
448     push dx
449     push si
450     mov cx, 1000                ; maximo de intentos
451 sf_try:
452     ; seed = seed * 25173 + 13849
453     mov ax, seed
454     mov bx, 25173
455     mul bx                      ; PRACTICA 5: MUL 16 bits -> DX:AX
456     add ax, 13849               ; PRACTICA 2: ADD/ADC = sumador total
457     adc dx, 0                   ; conservar el acarreo alto; seed usa solo AX
```

```
458     mov seed, ax
459     ; pos = seed % 400
460     xor dx, dx
461     mov bx, 400
462     div bx                      ; PRACTICA 5: DIV 16 bits -> DX=resto
463     mov si, dx
464     cmp byte ptr board[si], 0
465     jne sf_next
466     ; Colocar y dibujar
467     mov byte ptr board[si], 2
468     mov bx, si
469     mov al, '*'
470     mov ah, 0Ch
471     call DRAW_CELL
472     jmp sf_done
473 sf_next:
474     loop sf_try
475 sf_done:
476     pop si
477     pop dx
478     pop cx
479     pop bx
480     pop ax
481     ret
482 SPAWN_FOOD ENDP
483
484 ; ------------------------------------------------------------
485 ; DRAW_CELL: Dibuja un caracter en la celda BX del tablero
486 ; Entrada: BX = indice (0..399), AL = caracter, AH = color
487 ; Salida: ninguna
488 ; Registros modificados: AX, BX, CX, DX (se preservan)
489 ; ------------------------------------------------------------
490 DRAW_CELL PROC
491     push ax
492     push bx
493     push cx
494     push dx
495     push ax                     ; guardar caracter/color
496     ; fila = indice / 40, col = indice % 40 (DIV 8 bits)
497     mov ax, bx
498     mov cl, 40
499     div cl                      ; PRACTICA 5: DIV 8 bits -> AL=fila, AH=col
500     add al, 2                   ; fila pantalla = fila + 2
501     mov dh, al
502     add ah, 20                  ; col pantalla = col + 20
503     mov dl, ah
504     ; Posicionar cursor
505     mov ah, 02h
506     mov bh, 0
507     int 10h
508     ; Escribir caracter (AH=09h, BH=0, CX=1)
509     pop ax                      ; AL=car, AH=color
510     mov bl, ah
511     mov bh, 0
512     mov cx, 1
513     mov ah, 09h
514     int 10h
515     pop dx
516     pop cx
```

```
    pop bx
    pop ax
    ret
DRAW_CELL ENDP

; ---------------------------------------------------------
; PRINT_NUM: Imprime AX en decimal en la posicion actual del cursor
; Entrada: AX = numero (0..65535), cursor ya posicionado
; Salida: imprime digitos + 3 espacios
; Registros modificados: AX, BX, CX, DX (se preservan)
; ---------------------------------------------------------
PRINT_NUM PROC
    push ax
    push bx
    push cx
    push dx
    mov bx, 10
    xor cx, cx                  ; contador de digitos
    cmp ax, 0
    jne pn_loop
    xor dx, dx
    push dx                     ; caso especial: numero 0
    inc cx
    jmp pn_print
pn_loop:
    xor dx, dx
    div bx                      ; PRACTICA 5: DIV 16 bits -> DX=resto
    push dx                     ; apilar digito (PRACTICA 5: PUSH)
    inc cx
    cmp ax, 0
    jne pn_loop
pn_print:
    pop dx                      ; PRACTICA 5: POP
    add dl, '0'
    mov ah, 02h
    int 21h                     ; imprimir digito
    loop pn_print
    ; Limpiar con 3 espacios
    mov cx, 3
pn_sp:
    mov dl, ' '
    mov ah, 02h
    int 21h
    loop pn_sp
    pop dx
    pop cx
    pop bx
    pop ax
    ret
PRINT_NUM ENDP

DELAY PROC
    push ax
    push cx
    push dx
    mov ah, 86h
    mov cx, 0001h          ; CX:DX = microsegundos
    mov dx, 86A0h          ; 0001_86A0h = 100000 us = 100 ms
    int 15h
```

```
    pop dx
    pop cx
    pop ax
    ret
DELAY ENDP
; ---------------------------------------------------------------
; GAME_OVER: Muestra el mensaje GAME OVER y el puntaje, espera tecla
; Entrada: ninguna
; Salida: ninguna (el main loop termina)
; Registros modificados: AX, BX, CX, DX (se preservan)
; ---------------------------------------------------------------
GAME_OVER PROC
    push ax
    push bx
    push cx
    push dx
    ; Posicionar en (14, 19)
    mov ah, 02h
    mov bh, 0
    mov dh, 14
    mov dl, 19
    int 10h
    ; Imprimir el mensaje GAME OVER y el puntaje
    mov dx, offset msgGameOver
    mov ah, 09h
    int 21h
    ; Imprimir puntaje (cursor ya avanzado)
    mov ax, score
    call PRINT_NUM
    ; Esperar tecla
    mov ah, 00h
    int 16h
    pop dx
    pop cx
    pop bx
    pop ax
    ret
GAME_OVER ENDP

; PROGRAMA PRINCIPAL

inicio:
    call INIT_GAME
main_loop:
    call READ_KEY
    cmp al, 1
    je exit_game
    call MOVE_SNAKE
    cmp al, 1
    je game_over_h
    call DELAY
    jmp main_loop
game_over_h:
    call GAME_OVER
exit_game:
    ; Restaurar modo de video y cursor
    mov ax, 0003h
    int 10h
    mov ah, 01h
```

```
635     mov cx, 0607h
636     int 10h
637     ; Salir
638     mov ax, 4C00h
639     int 21h
640 
641 ; ============================================================
642 ; DATOS
643 ; ============================================================
644 BOARD_W      EQU 40
645 BOARD_H      EQU 10
646 BOARD_SIZE   EQU 400
647 
648 board        DB 400 dup(0)        ; 0=libre, 1=serpiente, 2=comida
649 queue        DW 400 dup(0)        ; indices de las celdas de la serpiente
650 head         DW 0                 ; proximo indice libre en queue
651 tail         DW 0                 ; indice de la cola en queue
652 dir          DB 1                 ; 0=arriba, 1=derecha, 2=abajo, 3=izquierda
653 headRow      DB 5
654 headCol      DB 21
655 score        DW 0
656 seed         DW 0
657 
658 msgScore     DB 'Puntaje: $'
659 msgGameOver DB 'GAME OVER! Puntaje: $'
```

Listing 14: snakeGame.asm

10.2. Tabla de variables y estructuras

La Tabla 11 repite la Tabla 4 en formato de varias páginas.

Cuadro 11: Constantes y variables (Anexo B).

Nombre	Tipo	Tamaño	Uso
BOARD_W	EQU	—	Ancho: 40.
BOARD_H	EQU	—	Alto: 10.
BOARD_SIZE	EQU	—	Celdas: 400.
board	DB	400 bytes	Estado de celdas (0 libre, 1 serpiente, 2 comida).
queue	DW	400 palabras	Cola circular de índices.
head	DW	1 palabra	Índice de escritura.
tail	DW	1 palabra	Índice de lectura.
dir	DB	1 byte	Dirección (0 a 3).
headRow	DB	1 byte	Fila de la cabeza.
headCol	DB	1 byte	Columna de la cabeza.
score	DW	1 palabra	Puntaje.
seed	DW	1 palabra	Semilla del generador, inicializada con los ticks del BIOS.
msgScore	DB	cadena	“Puntaje: ”.
msgGameOver	DB	cadena	“GAME OVER!”.

10.3. Tabla de servicios BIOS/DOS usados

La Tabla 12 lista los servicios del sistema.

Cuadro 12: Servicios BIOS y DOS empleados.

INT	AH	Función	Dónde se usa
`10h`	`01h`	Fijar forma del cursor (`CX=2000h` oculta; `CX=0607h` restaura).	`INIT_GAME`, salida.
`10h`	`00h`	Fijar modo texto 80x25.	`INIT_GAME`, salida.
`10h`	`02h`	Posicionar el cursor.	`DRAW_CELL`, `DRAW_BORDER`, `INIT_GAME`, `MOVE_SNAKE`.
`10h`	`09h`	Escribir carácter con atributo o repetirlo.	`DRAW_CELL`, `DRAW_BORDER`.
`16h`	`01h`	Consultar teclado sin bloquear.	`READ_KEY`.
`16h`	`00h`	Leer y consumir tecla.	`READ_KEY`, `GAME_OVER`.
`1Ah`	`00h`	Leer contador de ticks del BIOS.	`INIT_GAME`, semilla.
`15h`	`86h`	Espera BIOS en microsegundos, intervalo `CX:DX=0001:86A0h` (100 ms nominales).	`DELAY`.
`21h`	`02h`	Imprimir carácter en `DL`.	`PRINT_NUM`, salida.
`21h`	`09h`	Imprimir cadena terminada en `$`.	Mensajes de puntaje y fin de partida.
`21h`	`4Ch`	Terminar el programa.	Salida.

10.4. Referencias

Referencias

[1] EMU8086 Microprocessor Emulator, *Manual de usuario*.

[2] Brown, R. *Ralf Brown's Interrupt List* (INT 10h, INT 15h, INT 16h, INT 1Ah, INT 21h).

[3] Universidad Nacional de San Agustín de Arequipa, Escuela Profesional de Ingeniería de Sistemas. *Guía de prácticas de Arquitectura de Computadoras*.